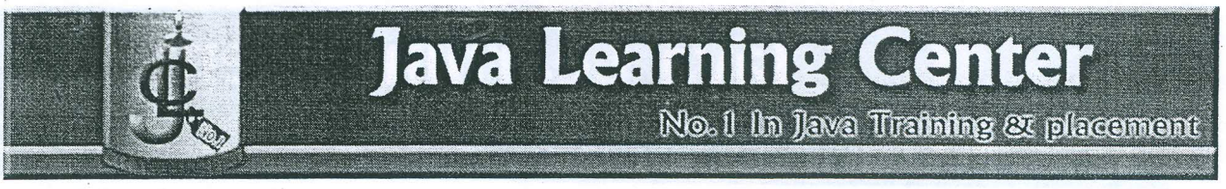
Java - 24



Java Learning Center

Reflection

Put Into Knowledge

Doc Version - 1.0

Author
Srinivas Dande
Kamlesh Kumar Singh

Introduction

- Java Reflection allows you
  - To inspect classes, interfaces, fields and methods at runtime.
  - To instantiate new objects.
  - To invoke methods dynamically at runtime.
  - To modify or access the values of fields.

- Java Reflection API is mainly used in:
  - IDE's (Integreted Development Environment) e.g. Eclipse, MyEclipse, NetBeans etc.
  - Debugger
  - Servers
  - Testing Tools etc.

- JVM creates default object of type java.lang.Class while loading the class.
- All types in Java including primitive types(int, long, float etc.) and arrays having default object of type java.lang.Class.
- You need to access that default object to inspect any classes or their members.

- You can use the following techniques to access default object:
  1. forName() method of java.lang.Class
  2. getClass() method of java.lang.Object
  3. the .class syntax

forName() method of java.lang.Class

- It is used to load the class dynamically and returns the instance of java.lang.Class.
- It should be used if you know the fully qualified name of class.
- It can be used only for user defined datatypes and not for primitive types.

Lab1364.java

```
package com.jlc.p1;
public class Lab1364 {
public static void main(String[] args) {
String className = "com.jlc.p1.Hello";
try {
Class cls = Class.forName(className);
System.out.println("Class loaded successfully");
System.out.println("Class Name :"+cls.getName());
} catch (Exception e) {
e.printStackTrace();
}       }       }

class Hello { }
```

getClass() method of java.lang.Object

- It returns the instance of java.lang.Class.
- It should be used if you have the object of class.
- It can be used only for user defined datatypes and not for primitive types.

Lab1365.java

```
package com.jlc.p1;
public class Lab1365 {
public static void main(String[] args) {
showClassInfo("OK");
showClassInfo(new Lab1365());
Hello h = new Hello();
showClassInfo(h);
}
static void showClassInfo(Object obj) {
Class cls = obj.getClass();
System.out.println("\nClass Name :" + cls.getName());
} }
class Hello { }
```

The .class syntax

- .class can be appended with any type name to get the object of java.lang.Class.
- It can be used for both user defined datatypes and primitive types.

Lab1366.java

```
package com.jlc.p1;
public class Lab1366 {
public static void main(String[] args) {
Class cls1 = Lab1366.class;
Class cls2 = int.class;
Class cls3 = boolean.class;
Class cls4 = Hello.class;
System.out.println("Name :" + cls1.getName());
System.out.println("Name :" + cls2.getName());
System.out.println("Name :" + cls3.getName());
System.out.println("Name :" + cls4.getName());
} }
class Hello { }
```

Methods	Description
public boolean isInterface()	determines if the specified Class object represents an interface type.
public boolean isArray()	determines if this Class object represents an array class.
public boolean isPrimitive()	determines if the specified Class object represents a primitive type.
public boolean isAnnotation()	determines if the specified Class object represents a Annotation type (From Java 5)
public boolean isAnonymousClass()	determines if the specified Class object represents a Anonymous type.
public boolean isEnum()	determines if the specified Class object represents an Enum.

Lab1367.java

```
public class Lab1367 {
public static void main(String[] args) {
Class cls1 = int.class;
Class cls2 = Cloneable.class;
Class cls3 = args.getClass();
Class cls4=Color.class;
verifyClass(cls1);
verifyClass(cls2);
verifyClass(cls3);
verifyClass(cls4);
}
static void verifyClass(Class cls) {
System.out.println("\n**Name :" + cls.getName());
System.out.println("Primitive :" + cls.isPrimitive());
System.out.println("Interface :" + cls.isInterface());
System.out.println("Array :" + cls.isArray());
System.out.println("Enum :" + cls.isEnum());
}
}
enum Color{
RED, BLUE
}
```

Methods	Description
public String getName()	Returns the fully qualified class name.
public String getSimpleName()	Returns the name of class.
public Package getPackage()	Returns the object of Package class.
public native Class getSuperclass();	Returns the default object of super class.
public native Class[] getInterfaces()	Returns the Class type array of implemented interfaces.

Lab1368.java

```
package com.jlc.ref;
import java.io.Serializable;
public class Lab1368{
public static void main(String[] args) {
try {
String cname = "com.jlc.ref.Student";
Class cl = Class.forName(cname);
showClassDetails(cl);
Class cl2 = "OK".getClass();
showClassDetails(cl2);
} catch (Exception e) {
e.printStackTrace();
}
}
static void showClassDetails(Class cl) {
System.out.println("\n**Name :" + cl.getName());
Class superclass = cl.getSuperclass();
System.out.println("Superclass :" + superclass.getName());
Package pack = cl.getPackage();
System.out.println("Package :" + pack.getName());
Class interfs[] = cl.getInterfaces();
System.out.println("Interfaces Implemented");
for (int i = 0; i < interfs.length; i++) {
Class intf = interfs[i];
System.out.println("\t" + intf.getName());
} } }
class Person implements Comparable {
public int compareTo(Object o) {
return 0;
}
}
class Student extends Person implements Serializable,
Cloneable { }
```

Methods	Description
public Constructor[] getConstructors()	Returns all the public Constructors
public Constructor[] getDeclaredConstructors()	Returns all the public and non-public Constructors
Method[] getMethods()	Returns all the public methods of current and super classes.
Method[] getDeclaredMethods()	Returns all the public and non-public methods of current class only.
Field[] getFields()	Returns all the public variables of current and super classes.
Field[] getDeclaredFields()	Returns all the public and non-public variables of current class only.
Class[] getClasses()	Returns all the public inner classes of current and super classes.
Class[] getDeclaredClasses()	Returns all the public and non-public inner classes of current class only.
Annotation[] getAnnotations()	Returns all annotations marked for current class.

Hai.java

```
package com.jlc.ref;
class Hello {
public String data;
int value;
public final void processData(int ab, String st) {}
boolean isValid(){return false;}
public class In1{}
class In2{}
}
public class Hai extends Hello{
String name;    public int id;
protected double fee;    private long phone;
private Hai(){}
private Hai(int ab,String st){}
public Hai(char ch,long val,boolean b1){}
Hai(char ch,double val,String st){}
public Hai(int ab,long val,float f1){}
protected Hai(byte by,boolean b1,float f2){}
void show(){}
public void display(int ab,boolean valid,long phn){}
private int[] getValues(String nm,long phn){return null;}
class A{}
public class B{}
class C{}
}
```

Lab1369.java

```
package com.jlc.ref;
import java.lang.reflect.*;
public class Lab1369{
public static void main(String[] args)  throws
Exception{
String cname = "com.jlc.ref.Hai";
Class cl=Class.forName(cname);
System.out.println("\n PUBLIC CONSTRUCTORS **");
Constructor[] pCons = cl.getConstructors();
for(int i=0;i<pCons.length;i++){
System.out.println(pCons[i]);
}
System.out.println("\n** DECLARED CONSTRUCTORS **");
Constructor[] aCons = cl.getDeclaredConstructors();
for(int i=0;i<aCons.length;i++){
System.out.println(aCons[i]);
}        } }
```

Lab1370.java

```
package com.jlc.ref;
import java.lang.reflect.*;
public class Lab1370 {
public static void main(String[] args)  throws
Exception{
String cname = "com.jlc.ref.Hai";
Class cl=Class.forName(cname);
System.out.println("\n** PUBLIC METHODS **");
Method[] pMthd = cl.getMethods();
for(int i=0;i<pMthd.length;i++){
Method m=pMthd[i];
System.out.println(m);
}
System.out.println("\n** DECLARED METHODS **");
Method[] aMthd = cl.getDeclaredMethods();
for(int i=0;i<aMthd.length;i++){
Method m=aMthd[i];
System.out.println(m);
}
} }
```

Lab1371.java

```
package com.jlc.ref;
import java.lang.reflect.*;
public class Lab1371 {
public static void main(String[] args)  throws Exception{
String cname = "com.jlc.ref.Hai";
Class cl=Class.forName(cname);
System.out.println("\n** PUBLIC FIELDS **");
Field[] pFlds = cl.getFields();
for(int i=0;i<pFlds.length;i++){
Field f=pFlds[i];
System.out.println(f);
}
System.out.println("\n** ALL DECLARED FIELDS **");
Field[] aFlds = cl.getDeclaredFields();
for(int i=0;i<aFlds.length;i++){
Field f=aFlds[i];
System.out.println(f);
}
}
}
```

Lab1372.java

```
package com.jlc.ref;
import java.lang.reflect.*;
public class Lab1372 {
public static void main(String[] args)  throws
Exception{
String cname = "com.jlc.ref.Hai";
Class cl=Class.forName(cname);
System.out.println("PUBLIC INNER CLASSES **");
Class[] pInCls = cl.getClasses();
for(int i=0;i<pInCls.length;i++){
Class c=pInCls[i];
System.out.println(c);
}
```

```
System.out.println("\n** DECLARED INNER CLASSES **");
Class[] aInCls = cl.getDeclaredClasses();
for(int i=0;i<aInCls.length;i++){
Class c=aInCls[i];
System.out.println(c);
}
}
}
```

Accessing Modifiers Information

- public int getModifiers() : To get the modifiers information for class, methods, fields etc.
- Constants are defined for all the modifiers in java.lang.reflect.Modifier class.
- getModifiers() method returns int value i.e sum of the all the constants of modifiers used.
- To verify the specific modifier used or not:
  - public static String toString(int mod);
  - public static boolean isXxx(int mod);
    Xxx is modifier name
    Ex: Public, Private, Protected, Abstract, Final, Static etc.

Lab1373.java

```
import java.lang.reflect.*;
public class Lab1373 {
public static void main(String[] args) {
try {
Class cl = Class.forName("JlcStudent");
System.out.println("PUBL :" + Modifier.PUBLIC);
System.out.println("PRIV :" + Modifier.PRIVATE);
System.out.println("PROT :" + Modifier.PROTECTED);
System.out.println("FINA :" + Modifier.FINAL);
System.out.println("STAT :" + Modifier.STATIC);
System.out.println("NATI :" + Modifier.NATIVE);
System.out.println("ABST :" + Modifier.ABSTRACT);
System.out.println("SYNC :" +
Modifier.SYNCHRONIZED);
Method ms[] = cl.getDeclaredMethods();
for (int i = 0; i < ms.length; i++) {
Method m = ms[i];
System.out.println("\n***** " + m);
int mod = m.getModifiers();
System.out.println("Modifier :" + mod);
String str = Modifier.toString(mod);
System.out.println("STR :" + str);
System.out.println("PUBLIC :" + Modifier.isPublic(mod));
System.out.println("PRIVATe :" +
Modifier.isPrivate(mod));
System.out.println("PROTECTEd :" +
Modifier.isProtected(mod));
System.out.println("FINAL :" + Modifier.isFinal(mod));
System.out.println("STATIC :" + Modifier.isStatic(mod));
System.out.println("NATIVE :" + Modifier.isNative(mod));
System.out.println("ABSTTRACT :" +
Modifier.isAbstract(mod));
System.out.println("SYNCHRONIZED :" +
Modifier.isSynchronized(mod));
}
} catch (Exception e) {
System.out.println(" JlcStudent class not found ");
e.printStackTrace();
}
}
}
abstract class JlcStudent {
public void m1() { }
native final void m2();
public synchronized void m3() { }
protected abstract void m4();
private static final void m5() { }
void m6() { }
}
```

Modifier.public — will print the integer value (default value)

Modifier.toString() Pass some value according to value we call get the Modifier Name

Creating object using default constructor

public Object newInstance()

Lab1374.java

```
package com.jlc.ref;
public class Lab1374 {
public static void main(String[] args) {
try {
String cname = "com.jlc.ref.User";
Class cl = Class.forName(cname);
Object obj = cl.newInstance();
System.out.println(obj);
} catch (Exception e) {
e.printStackTrace();
} } }
```

```
class User {
User() {
System.out.println("** User Def Cons ");
}
}
```

- If you know the parameter types of the constructor you want to access, you can use following rather than obtain the array all constructors.
  - Constructor c = cl.getDeclaredConstructor(Class[] paramsTypes);
- It will throw java.lang.NoSuchMethodException if required constructor not found

Lab1375.java

```
package com.jlc.ref;
import java.lang.reflect.*;
public class Lab1375 {
public static void main(String[] args) {
try {
String cname = "com.jlc.ref.Emp";
Class cl = Class.forName(cname);
Class paramsReq[] = new Class[] { int.class,
String.class,
int.class, long.class };
Constructor c =
cl.getDeclaredConstructor(paramsReq);
System.out.println(c);
Class paramsReq1[] = new Class[] { String.class,
int.class };
Constructor c1 =
cl.getDeclaredConstructor(paramsReq1);
System.out.println(c1);
} catch (ClassNotFoundException e) {
e.printStackTrace();
} catch (NoSuchMethodException e) {
System.out.println("\n** Required Constructor not
found");
}
}
}
class Emp {
Emp(int eid, String nm, int age, long phone) { }
Emp(int eid, String nm) { }
}
```

Creating object using parameterized constructors

public Object newInstance(Object[] args)

Lab1376.java

```
package com.jlc.ref;
import java.io.DataInputStream;
import java.lang.reflect.Constructor;
public class Lab1376 {
public static void main(String[] args) {
DataInputStream dis = null;
try {
dis = new DataInputStream(System.in);
String cname = "com.jlc.ref.Student";
Class cl = Class.forName(cname);
Constructor cons[] = cl.getConstructors();
Object consArgs[] = null;
for (int i = 0; i < cons.length; i++) {
Constructor c = cons[i];
System.out.println("\n** " + c);
Class params[] = c.getParameterTypes();
consArgs = new Object[params.length];
for (int j = 0; j < params.length; j++) {
String type = params[j].getName();
System.out.println("Enter value of " + type + "
type");
String data = dis.readLine();
if (type.equals("boolean")) {
consArgs[j] = new Boolean(data);
} else if (type.equals("byte")) {
consArgs[j] = new Byte(data);
} else if (type.equals("short")) {
consArgs[j] = new Short(data);
} else if (type.equals("int")) {
consArgs[j] = new Integer(data);
} else if (type.equals("long")) {
consArgs[j] = new Long(data);
} else if (type.equals("float")) {
consArgs[j] = new Float(data);
} else if (type.equals("double")) {
consArgs[j] = new Double(data);
} else if (type.equals("char")) {
consArgs[j] = new Character(data.charAt(0));
} else if (type.equals("java.lang.String")) {
consArgs[j] = data;
}
}
Object obj = c.newInstance(consArgs);
System.out.println("In main obj :" + obj);
}
} catch (Exception e) {
e.printStackTrace();
} } }

class Student {
public Student() {
System.out.println("\nStudent() called -- :" + this);
}

public Student(int sid, String name, long phone) {
System.out.println("\nStudent(int sid, String name,
long phone) called -- :"+ this);
System.out.println("Sid :" + sid);
System.out.println("Name :" + name);
System.out.println("Phone :" + phone);
}

public Student(boolean reg, char ch, double fee) {
System.out.println("\nStudent(boolean reg, char
ch,double fee) called -- :"+ this);
System.out.println("Reg :" + reg);
System.out.println("Ch :" + ch);
System.out.println("Fee :" + fee);
}
}
```

Invoking Method Dynamically

public Object invoke(Object objReq , Object[] argsToMethod)

- The return type of the invoke() is Object type that contains the result of the method invoked.
- The first parameter of the invoke() is the object of your class required to invoke your method. It can be null in the case of static method.

- The second parameter of the invoke() is Object array. It contains the values requires as the argument for your method (if your method is parameterized). For the primitive type you need to store the corresponding Wrapper class object.
- The data in the Object array should be stored as same number, type and order as the parameter required for your method.

Lab1377.java

```
package com.jlc.ref;
import java.io.*;
import java.lang.reflect.*;
import java.util.*;
public class Lab1377 {
public static void main(String[] args) throws
Exception {
Scanner sc=new Scanner(System.in);
System.out.println("Enter fully qualified class
name :");
String cName=sc.nextLine();
Class cl1 = Class.forName(cName);
DataInputStream dis = new
DataInputStream(System.in);
Method ms[] = cl1.getDeclaredMethods();
for (int i = 0; i < ms.length; i++) {
Method m = ms[i];
m.setAccessible(true);
System.out.println("\nCalling the method " +
m.getName());
Object objReq = null;
int mod = m.getModifiers();
boolean st = Modifier.isStatic(mod);
if (!st) {
objReq = cl1.newInstance();
}
Class params[] = m.getParameterTypes();
Object[] reqArgs = new Object[params.length];
for (int j = 0; j < params.length; j++) {
String type = params[j].getName();
System.out.println("Enter Value of type\t:" +
type);
String val = dis.readLine();
if (type.equals("boolean"))
reqArgs[j] = new Boolean(val);
else if (type.equals("byte"))
reqArgs[j] = new Byte(val);
else if (type.equals("char"))
reqArgs[j] = new Character(val.charAt(0));
else if (type.equals("short"))
reqArgs[j] = new Short(val);
else if (type.equals("int"))
reqArgs[j] = new Integer(val);
else if (type.equals("long"))
reqArgs[j] = new Long(val);
else if (type.equals("float"))
reqArgs[j] = new Float(val);
else if (type.equals("double"))
reqArgs[j] = new Double(val);
else if (type.equals("java.lang.String"))
reqArgs[j] = val;
}
Object res = m.invoke(objReq, reqArgs);
System.out.println("Result is\t:" + res);
}}}

class Student {
private void showDetails() {
System.out.println("-- showDetails () of Student\t:" +
this);
}
static int searchId(int ab, String bc, boolean b1) {
System.out.println("-- searchId () of Student\t:");
System.out.println("ab\t:" + ab);
System.out.println("bc\t:" + bc);
System.out.println("b1\t:" + b1);
return 99;
}
boolean verifyStudent(char ch, String bc, boolean b1,
long val, float f1) {
System.out.println("-- verifyStudent () of Student\t:" +
this);
System.out.println("ch\t:" + ch);
System.out.println("bc\t:" + bc);
System.out.println("b1\t:" + b1);
System.out.println("val\t:" + val);
System.out.println("f1\t:" + f1);
return true;
}}
```


m.setAccessible(true)
→ using this method you can access private member also.

Accessing and modifying the variables or fields

Field field = cl.getField("someFieldName");

Accessing private members or non-accessible members

void setAccessible(boolean flag) throws SecurityException

if flag is true then the member can be accessed.

Lab1378.java

```
package org.sd.ref;
import java.io.DataInputStream;
import java.lang.reflect.Field;
public class Lab1378 {
public static void main(String[] args) {
DataInputStream dis = null;
try {
dis = new DataInputStream(System.in);
String cname = "org.sd.ref.User";
Class cl = Class.forName(cname);
Object usr = cl.newInstance();
Field flds[] = cl.getDeclaredFields();
for (int i = 0; i < flds.length; i++) {
Field f = flds[i];
f.setAccessible(true);
String nm = f.getName();
String ty = f.getType().getName();
System.out.println("Enter value for " + nm + " of
type " + ty);
String data = dis.readLine();
Object value = null;
if (ty.equals("boolean")) {
value = new Boolean(data);
} else if (ty.equals("byte")) {
value = new Byte(data);
} else if (ty.equals("short")) {
value = new Short(data);
}
else if (ty.equals("int")) {
value = new Integer(data);
} else if (ty.equals("long")) {
value = new Long(data);
}else if (ty.equals("float")) {
value = new Float(data);
} else if (ty.equals("double")) {
value = new Double(data);
} else if (ty.equals("char")) {
value = new Character(data.charAt(0));
} else if (ty.equals("java.lang.String")) {
value = data;
}
f.set(usr, value);
}
System.out.println("\nObject is :" + usr);
} catch (Exception e) {
e.printStackTrace();
}}}

class User {
private int uid;
private String uname;
private String password;
public String toString() {
return uid + "\t" + uname + "\t" + password;
}
}
```

Modifying content of String class

- String has a private member with the name char[] value.
- You can get this char[] type value field and modify the data using reflection.

Lab1379.java

```
package com.jlc.ref;
import java.lang.reflect.*;
public class Lab1379 {
public static void main(String[] args) throws
Exception{
String str="ABC";
System.out.println(str);
Util.changeStringValue(str,"JLC");
System.out.println(str);
}
}

class Util{
static void changeStringValue(String str,String
val) throws Exception{
Class cl=str.getClass();
Field fld=cl.getDeclaredField("value");
fld.setAccessible(true);
Object obj=fld.get(str);
char chrs[]=(char[])obj;
for(int i=0;i<chrs.length;i++){
chrs[i]=val.charAt(i);
}
}
}
```